

Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине: «Аппаратно-программное обеспечение ЭВМ и сетей»

Тема: «Изучение аппаратно-программной архитектуры процессоров семейства
Intel – разработка ассемблерной вставки»

Выполнил:

Студент 4 курса

Группы АС-66

Гончерёнок К.А.

Проверила:

Кулик А.Д.

Брест 2026

Цель работы: изучение программной модели микропроцессора, регистров, организации памяти; практическая работа с ассемблерными вставками.

Вариант 4

Задание: 1. Изучить следующие теоретические сведения: программная модель микропроцессора; регистры и их назначение; организация памяти; основные конструкции и базовые команды ассемблера (используя материалы лекций и методические указания).

2. Написать ассемблерную вставку, реализующую обработку строки согласно варианту, выданному преподавателем. Оформить ее в виде отдельной функции.

3. Реализовать данную обработку строки также в виде функции на C++. Сравнить быстродействие обоих вариантов.

Вариант задания: даны 2 строки. Совместить четные символы одной строки с нечетными другой.

```
Платформа: x86 (32-bit)

Строка A = bxgdxgxsxkx
Строка B = хухаххххi
Результат C++ : bygagashki
Результат ASM : bygagashki

Время C++ (1000000 итераций): 5527214 мкс
Время ASM (1000000 итераций): 136573 мкс

Результаты совпадают.

D:\apoevms\laba1\Debug\laba1.exe (процесс 124
```

Код программы:

```
#include <iostream>
#include <string>
#include <chrono>
#include <locale>
#include <clocale>
//функция на c++
std::string combineCpp(const std::string& a, const std::string& b) {
    std::string result;
    result.reserve(a.size() + b.size());

    size_t i = 0;
    size_t j = 1;

    while (i < a.size() || j < b.size()) {
        if (i < a.size()) { result += a[i]; i += 2; }
        if (j < b.size()) { result += b[j]; j += 2; }
    }
    return result;
}
//ассемблерная вставка
void combineAsm(const char* a, const char* b, char* out) {
    __asm {
        push esi
        push edi
        push ebx

        mov esi, a
        mov ebx, b
        mov edi, out
        xor ecx, ecx
```

```

    mov     edx, 1

loopA:
    mov     al, byte ptr[esi + ecx]
    test    al, al
    jz      loopB
    mov     byte ptr[edi], al
    inc     edi
    add     ecx, 2

    loopB:
    mov     al, byte ptr[ebx + edx]
    test    al, al
    jz      checkA
    mov     byte ptr[edi], al
    inc     edi
    add     edx, 2

    checkA:
    mov     al, byte ptr[esi + ecx]
    test    al, al
    jnz     loopA

    mov     al, byte ptr[ebx + edx]
    test    al, al
    jnz     loopB

    done :
    mov     byte ptr[edi], 0

    pop     ebx
    pop     edi
    pop     esi
}

}

int main() {
    setlocale(LC_ALL, "Russian");

#ifdef _M_IX86
    std::cout << "Платформа: x86 (32-bit)\n\n";
#elif defined(_M_X64)
    std::cout << "Платформа: x64 - переключи на x86!\n";
    return 1;
#endif

    std::string a = "bxgxxsxkx";
    std::string b = "xyxaxhxi";

    auto t1 = std::chrono::high_resolution_clock::now();
    std::string resCpp;
    for (int k = 0; k < 1000000; ++k)
        resCpp = combineCpp(a, b);
    auto t2 = std::chrono::high_resolution_clock::now();

    char buffer[256] = { 0 };
    auto t3 = std::chrono::high_resolution_clock::now();
    for (int k = 0; k < 1000000; ++k)
        combineAsm(a.c_str(), b.c_str(), buffer);
    auto t4 = std::chrono::high_resolution_clock::now();

    std::cout << "Строка A = " << a << "\n";
    std::cout << "Строка B = " << b << "\n";
    std::cout << "Результат C++ : " << resCpp << "\n";
    std::cout << "Результат ASM : " << buffer << "\n\n";

    auto cppTime = std::chrono::duration_cast<std::chrono::microseconds>(t2 - t1).count();
    auto asmTime = std::chrono::duration_cast<std::chrono::microseconds>(t4 - t3).count();

    std::cout << "Время C++ (1000000 итераций): " << cppTime << " мкс\n";
    std::cout << "Время ASM (1000000 итераций): " << asmTime << " мкс\n";

    if (resCpp != buffer) {
        std::cout << "\nРезультаты не совпадают!\n";
        return 2;
    }
}

```

```
    }  
    std::cout << "\nРезультаты совпадают.\n";  
    return 0;  
}
```

Результат работы программы:

Вывод: В ходе работы изучены программная модель микропроцессора, назначение регистров и основы работы с памятью. Реализованы две функции для совмещения чётных символов одной строки с нечётными другой: на ассемблере (`__asm`-вставка) и на C++. Обе дали одинаковый результат, что говорит о корректности обеих реализаций. Замеры показали, что ассемблерный вариант работает примерно в 2,5 раза быстрее: в C++ чётность проверяется через `i % 2`, а в ассемблере — одной командой `test`, плюс прямой доступ к памяти через регистры без лишних накладных расходов. При этом C++ выигрывает в читаемости и переносимости, поэтому ассемблер оправдан только там, где важна максимальная скорость.